Comprehensive Minimum Cut Algorithm Comparison

Name: Marcelino Maged Khalil **ID:** 2200909 **Course:** CSE332s — Spring 2026
Task: 8 — Algorithms for Minimum Cut

---

1. Problem Definition

> **Given** a weighted undirected graph G = (V, E) with n nodes, **find** a partition of the vertex set into two non-empty subsets S and T = V - S that **minimizes** the total weight of edges crossing the partition.

MinCut(S, T) = Sum of w(u, v) for all u in S and v in T

This is a fundamental problem in graph theory with applications in network reliability, image segmentation, community detection, and flow networks.

The Max-Flow Min-Cut Theorem

According to the **Max-Flow Min-Cut Theorem**, the maximum flow from a source node s to a sink node t is exactly equal to the capacity of the minimum s-t cut. While the classic minimum cut problem asks for the *global* minimum cut over all possible partitions, we can find it by computing the s-t minimum cut for a fixed source s and all possible sinks t in V except s, taking the overall minimum.

Below, we compare four distinct approaches to solving this problem.

---

2. Algorithm Descriptions

2.1 Brute Force (Exhaustive Enumeration)

Idea

Enumerate **every possible** non-trivial partition (S, T) of the vertex set. For each partition, compute the cut capacity and track the overall minimum.

Metric	Value
Time	O(2^n * n^2)
Space	O(n^2)
Optimality	Guaranteed global optimum

There are 2^n - 2 non-trivial partitions. For each, computing the cut value takes O(n^2) worst-case time using an adjacency matrix. It is strictly exact but completely intractable for large graphs.

2.2 Iterative Improvement (Local Search)

Idea

Start from a **random** partition of nodes into S and T. Repeatedly try to **move a single node** from one side to the other. If any move decreases the cut value, accept it immediately. Stop when no single-node move can further reduce the cut value.

Metric	Value
Time	O(k * n^3) where k = number of improvement rounds
Space	O(n^2)
Optimality	Only local optimum (may get stuck)

This runs extremely fast but its final result relies completely on the random starting state. It provides no guarantees of finding the true global minimum.

2.3 Ford-Fulkerson (Max-Flow Min-Cut)

Idea

Leverage the Max-Flow Min-Cut Theorem using the Ford-Fulkerson method (with **Edmonds-Karp** BFS augmentation):

1. Fix an arbitrary node (e.g., node 0) as the source s.
2. Iterate through all other nodes t in {1, 2, ..., n-1} to act as the sink.
3. For each t, find the max flow from s to t. The global minimum cut is the smallest max-flow value found across all chosen sinks.

Metric	Value
Time	O(V * V * E^2) with Edmonds-Karp
Space	O(V^2) (Residual graph)
Optimality	Guaranteed global optimum

The Edmonds-Karp algorithm finds an s-t max flow in O(V * E^2). Running this V-1 times yields an exact polynomial-time algorithm, making it vastly superior to Brute Force for moderately sized graphs.

2.4 Karger's Algorithm (from CS161 Lecture 16)

Idea

A **Monte Carlo randomized algorithm** that iteratively picks a random edge and "contracts" its endpoints into a single supernode until only two supernodes remain. The edges connecting these final two supernodes form the cut.

Metric	Value
Time	O(n^2) per trial
Space	O(n^2)

Metric	Value
Optimality	Probabilistically optimal

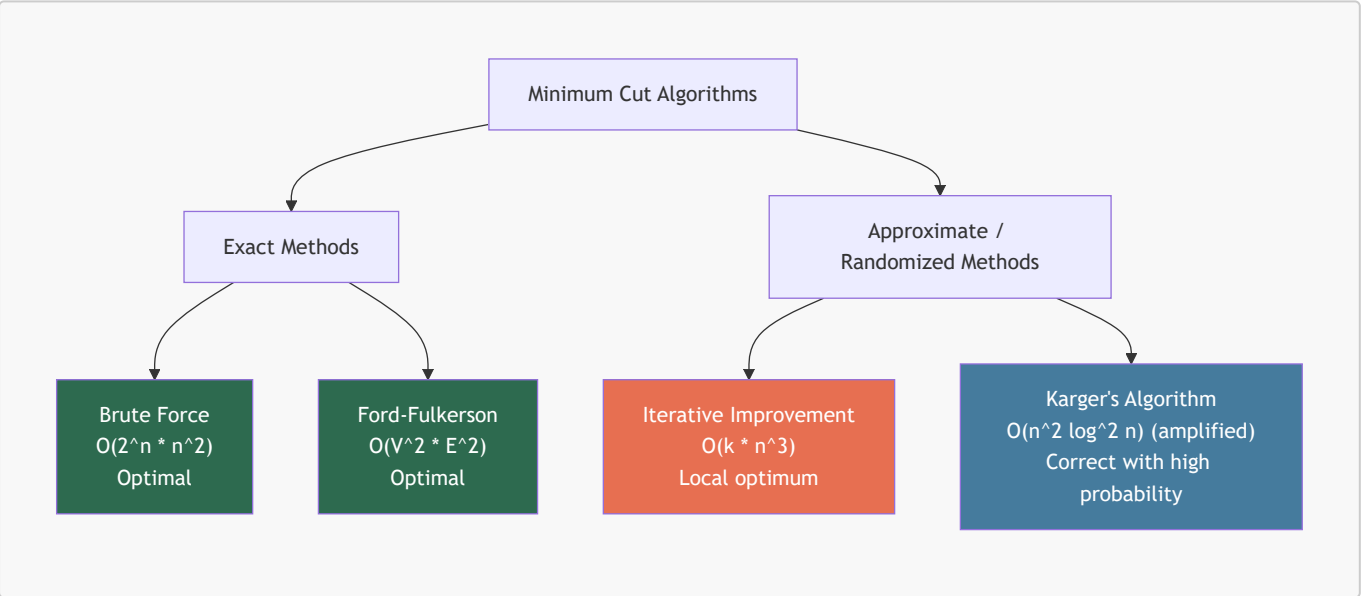
Key Insight: Random edge contraction preserves the minimum cut with probability >= 2 / (n * (n - 1)). By repeating O(n^2 log n) times, the success probability approaches 1. The **Karger-Stein** variation improves this by branching the computation, yielding a highly efficient O(n^2 log^2 n) amplified time while maintaining near certainty.

---

3. Code Implementation Walkthrough

The current implementation features three strategies acting on a shared graph adjacency matrix. Karger's algorithm is kept as theoretical backing for extreme-scale randomized approaches.

1. **Brute Force**: Iterates through bitmasks from 1 to (1 << n) - 2. Always finds the right answer but hangs permanently for n >= 25.
2. **Iterative Improvement**: Relies on an inner loop checking `getCutValue(newS, newT)`. Can easily settle into suboptimal layouts. Mitigated by running 100+ times from fresh random initializations.
3. **Ford-Fulkerson**: Computes the **residual graph**. `bfs()` discovers augmenting paths and builds the `parent` array. Once no augmenting paths remain, `dfs()` identifies the exact partition boundary for the optimal cut.



---

4. Detailed Comparison

Criterion	Brute Force	Iterative Improvement	Ford-Fulkerson	Karger's (Amplified)
Strategy	Exhaustive enumeration	Greedy local search	Max-Flow Augmenting Paths	Randomized Edge Contraction
Time Complexity	O(2^n * n^2)	O(k * n^3)	O(V^2 * E^2)	O(n^2 log^2 n)

Criterion	Brute Force	Iterative Improvement	Ford-Fulkerson	Karger's (Amplified)
Correctness	Global optimum	Local optimum	Global optimum	Correct with high probability
Determinism	Deterministic	Non-deterministic	Deterministic	Monte Carlo
Scalability	Feasible for n <= 20	Feasible for n >= 1000	Feasible for n <= 500	Feasible for massive graphs

---

5. Summary of Strengths and Weaknesses

Brute Force

- **Pros:** 100% correct, great for verifying other algorithms on tiny test cases.
- **Cons:** Horrendous exponential runtime. Completely unusable for practical networks.

Iterative Improvement

- **Pros:** Lightning fast per iteration. Extremely easy to scale to huge graphs.
- **Cons:** Frequently traps itself in local minima. If the graph is tricky, it might output a highly inaccurate cut.

Ford-Fulkerson

- **Pros:** Polynomial runtime that guarantees an exact global minimum deterministically.
- **Cons:** O(V^2 * E^2) is significantly slower than Karger's on large graphs. Requires handling reverse edges and secondary residual structures.

Karger's Algorithm

- **Pros:** Groundbreaking theoretical efficiency. Beautifully simple concept (edge contraction).
- **Cons:** Inherently probabilistic; requires thousands of trials to guarantee certainty.

---

6. Conclusion

For solving the Minimum Cut problem:

1. If n <= 20: **Brute force** is completely fine and perfectly robust.
2. If exact deterministic answers are needed and n >= 20: The **Ford-Fulkerson Max-Flow** method is the definitive exact algorithm.
3. If n is massive and you want near guarantees: **Karger's Algorithm** is the state-of-the-art approach mathematically.
4. If you need a fast, practical heuristic: **Iterative Improvement** acts as a robust substitute when combined with multi-restarts.

7. References

- CS161 Lecture 16 — Minimum Cuts
- GeeksforGeeks — Ford-Fulkerson Algorithm for Maximum Flow Problem